

MACHINE LEARNING OPERATIONS

Assignment 01

Diabetes Prediction on AWS EKS

Module	Machine Learning Operations
Assignment Type	Individual Assignment
Submission	Source Code + Deployment Evidence + Short Report
Total Marks	100

Background

This assignment uses iam-veeramalla/first-mlops-project as the application baseline. Run its training and FastAPI code locally, build a Docker image, publish it to private Amazon ECR, and deploy it on Amazon EKS in Mumbai (ap-south-1). Use one managed worker node and one application replica, with EKS Auto Mode disabled. The workflow extends the tutorial with cloud deployment, evaluation and cost tracking. Keep the original author attribution and document all changes to the supplied code.

Task 1: Data Preparation

1. Clone <https://github.com/iam-veeramalla/first-mlops-project.git> and record the commit using git rev-parse HEAD. Inspect the dataset URL and feature selection in train.py.
2. Use Outcome as the target: 1 represents the diabetes class and 0 represents the non-diabetes class. Select Pregnancies, Glucose, BloodPressure, BMI and Age as inputs.
3. Report the number of observations, data types, class distribution and summary statistics. Check for missing values, duplicate rows and implausible measurements.
4. Document any cleaning decisions. If you introduce imputation, fit it on training data only and reuse it at prediction time. Do not treat every zero value as missing.
5. Create an 80–20 train-test split with random_state=42. Keep the test set separate from model fitting and record whether you use stratification.

Task 2: Model Development

1. Use the repository as the working directory. Create a Python environment, install requirements.txt, and record the Python and package versions.
2. Run python train.py to train the supplied Random Forest. Add a fixed model random seed for repeatability and document this change to the baseline.
3. Save the trained model as diabetes_model.pkl using joblib. Reload it and confirm that it produces the same predictions for a fixed test sample.
4. Explain the difference between training and inference, and why the API must use the same feature names, order and preprocessing as training.

Task 3: Model Evaluation

The following evaluation requirements extend the deployment tutorial into an assessed assignment. Calculate results on the held-out test set; do not enter assumed or example scores.

1. Report accuracy, precision, recall, F1-score and ROC-AUC. Treat Outcome=1 as the positive class and use positive-class probabilities for ROC-AUC.
2. Generate a labelled confusion matrix and identify true positives, true negatives, false positives and false negatives. Explain the errors in the context of this dataset.
3. Plot the ROC curve and discuss the model's limitations. State why test performance alone does not establish suitability for clinical use.

Present the results in a table similar to the following:

Evaluation Metric	Random Forest	Interpretation
Accuracy		
Precision		
Recall		
F1-Score		
ROC-AUC		

Task 4: API Development and Testing

1. Review and run the supplied main.py. Verify that FastAPI loads diabetes_model.pkl at startup and validates the five input fields through its Pydantic model.
2. Verify the existing GET / response and POST /predict endpoint, which returns the predicted class through the diabetic Boolean field.
3. Run the API locally with Uvicorn and open /docs. Submit a valid request, capture the actual response and compare it with direct model inference.
4. Test a request with a missing field and a request with an invalid field type. Record the status codes and explain any additional range validation you implement.

Example request values: Pregnancies=2, Glucose=130, BloodPressure=70, BMI=28.5, Age=45. Report the observed prediction; these values do not prescribe a result.

Task 5: Docker Containerization

1. Review the Dockerfile. Explain its base image, working directory, dependency installation, copied files and Uvicorn startup command.
2. Train and save the model before building the image. Ensure the image includes the model artifact and everything required for inference.
3. Build the image, run it with host port 8000 mapped to container port 8000, and repeat the local API tests. Save the commands and evidence.
4. Compare the local and container responses for the same inputs. Record the image tag and explain how a container improves reproducibility.

Task 6: Publish the Image to Amazon ECR

Selected workflow: local training and testing, Docker image creation, private Amazon ECR publication, then Amazon EKS deployment. Use ap-south-1 for both ECR and EKS.

1. Build an image compatible with the EKS worker architecture. For x86 workers use linux/amd64, including on Apple Silicon. Use a versioned tag such as v1.
2. Create a private ECR repository named diabetes-api. Authenticate Docker with `aws ecr get-login-password --region ap-south-1` and `docker login --username AWS --password-stdin` against your ECR registry.
3. Tag and push the image as `ACCOUNT_ID.dkr.ecr.ap-south-1.amazonaws.com/diabetes-api:v1`. Replace `ACCOUNT_ID` with your actual AWS account ID; record the image digest.
4. Confirm the image in ECR and test a pull. Keep AWS credentials out of the build context. Add a `.dockerignore` excluding the local environment and `.git`, while retaining `diabetes_model.pkl`.
5. Give the managed worker node IAM role the required ECR image-pull permissions. Confirm network connectivity from the nodes to ECR and the services used for image downloads.

Task 7: Deploy and Verify on Amazon EKS

1. Before provisioning, check AWS account plan, service access, remaining credits and regional prices. Prepare AWS CLI, kubectl and your selected cluster provisioning tool; verify the active AWS identity.
2. Select Custom configuration and disable EKS Auto Mode. Name the cluster diabetes-mlops, choose a standard-support version, and create a managed node group with desired/minimum/maximum size 1. Size the instance for system Pods and the model.
3. Configure the VPC and subnets across the required Availability Zones. Ensure nodes can reach the API, registry and required AWS services. Explicitly review any NAT gateway or VPC endpoint costs.
4. Configure cluster access and update kubeconfig. Verify that nodes are Ready. Keep the registry Region aligned with the cluster when using private ECR.
5. Edit `k8s-deploy.yml`: replace the Docker Hub image placeholder with the ECR URI, change replicas from 2 to 1, and change Service type from LoadBalancer to ClusterIP. Keep Service port 80 and targetPort 8000. Add resource limits and health probes.
6. Run `kubectl apply -f k8s-deploy.yml` and check rollout status for deployment/diabetes-api. Run `kubectl port-forward service/diabetes-api-service 8000:80`, then test `localhost:8000/docs` and `/predict`.
7. Compare the same request across local Python, Docker and EKS. Capture responses and deployment evidence. Delete one application Pod and document its replacement.
8. After collecting evidence, remove application resources, node groups and the cluster. Verify removal of lab-created load balancers, volumes, NAT gateways and unused public IP allocations; remove registry images if no longer needed.

Selected lab configuration: one managed worker node, one application replica, EKS Auto Mode disabled, no optional paid EKS capabilities, and no external load balancer. This is a learning setup with limited availability. Port forwarding does not remove node networking or cluster charges.

AWS Free Tier and Cost Planning

EKS has no ongoing free cluster allowance. A short lab may have no out-of-pocket cost if valid credits cover all eligible usage. Free account plans expose only selected services; confirm EKS access in your account before provisioning. A Paid plan may be required, and charges beyond applicable credits are payable.

Project budget assumption: a recently created AWS Free account with \$100 remaining credits, as reported at planning time. Recheck the balance, expiry and eligible services before deployment. Opening the EKS creation form does not establish successful provisioning or credit coverage. If a Paid plan is required, remaining eligible credits carry over; usage beyond them can be billed. [3]

Cost examples before credits and tax: the standard EKS control plane is \$0.10/hour, so 4 hours costs \$0.40 and 730 hours costs \$73. These amounts exclude EC2 workers, EBS, IPv4, networking and other services. Extended-support versions cost \$0.60/hour for the control plane. [1]

ECR advertises 500 MB/month of private storage for one year for new ECR customers; check account eligibility. Larger images and retained versions can exceed this. Use private ECR for this project and remove unused image versions after testing. [2]

Estimate the complete lab in AWS Pricing Calculator, create budget alerts and set a cleanup time. Alerts are not a spending cap. Deleting Pods or scaling workers to zero does not stop the EKS control-plane charge; delete the cluster when the lab is complete. [1, 5]

Task 8: Record Costs and Clean Up

1. Prepare an estimate with separate lines for the cluster, worker nodes, disk storage, image storage, networking and any optional services. State the Region and planned running hours.
2. Record your applicable credit balance and expiry, plus any Free Tier allowances. Do not include account secrets in screenshots.
3. Choose a short test window and verify deletion of resources after the EKS tests. Review Billing for usage as it becomes available, including retained registry storage.
4. Explain whether the project was covered by credits or incurred charges. If no suitable credits are available, complete local testing first and provision EKS only for the cloud demonstration.

Expected Deliverables

Students should submit the following:

1. Source package: repository URL and commit ID, train.py, main.py, requirements.txt, setup instructions and a clear list of changes to the supplied code.
2. Data and evaluation report: dataset inspection, preprocessing decisions, split details, measured metrics, confusion matrix and ROC curve.
3. Model artifact: the saved model or clear instructions to regenerate it, including the input feature order and any fitted preprocessing.
4. Cloud deployment package: Dockerfile, .dockerignore, modified k8s-deploy.yml, EKS settings, ECR URI and digest, plus build, push and deployment commands.
5. Verification evidence: labelled screenshots or command outputs for local API tests, container tests, Kubernetes resources and prediction responses.
6. Short report: architecture, component explanations, issues and fixes, cost estimate, credit eligibility, cleanup evidence, limitations and references.

Submission Notes

- Include meaningful comments and explain the major steps in your own words.
- Label every plot, table and screenshot. Distinguish measured results from expected behaviour.
- Keep credentials out of submitted files. Provide instructions another student can follow.
- Cite the tutorial, companion code and dataset; identify improvements you add to the baseline.

References

Tutorial: Abhishek Veeramalla, Build and Deploy your First MLOps project in 40 minutes to Kubernetes.
<https://www.youtube.com/watch?v=hw17NDhjVHo>

Companion project: <https://github.com/iam-veeramalla/first-mlops-project>

Dataset: <https://raw.githubusercontent.com/plotly/datasets/master/diabetes.csv>

[1] Amazon EKS pricing: <https://aws.amazon.com/eks/pricing/>

[2] Amazon ECR pricing: <https://aws.amazon.com/ecr/pricing/>

[3] AWS account plans: <https://docs.aws.amazon.com/awsaccountbilling/latest/aboutv2/free-tier-plans.html>

[4] Deployment baseline: <https://github.com/iam-veeramalla/first-mlops-project/blob/main/k8s-deploy.yml>

[5] EKS cleanup: <https://docs.aws.amazon.com/eks/latest/userguide/delete-cluster.html>

Pricing and plan information checked on October 1, 2026. Verify current rates and account eligibility before creating resources.